CITY PULSE – CIVIC ISSUE REPORTING AND TRACKING SYSTEM

A PROJECT REPORT

Submitted by

S BALAJI

2116240701069

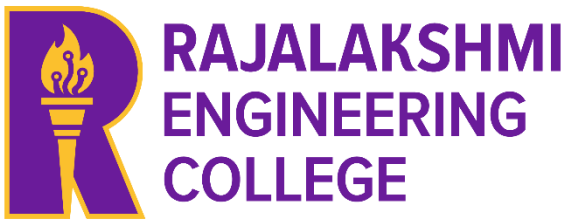
in partial fulfilment of the End Semester Examination for the course

CB23F35 –

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF ENGINEERING
COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

NOVEMBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled "**CITY PULSE – CIVIC ISSUE REPORTING AND TRACKING SYSTEM**" is the bonafide work of S BALAJI **[2116240701069],** who carried out the individual project work on the Zoho Creator Low-Code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator during the academic year 2026–2027, Semester 5, under the supervision.

SIGNATURE

Dr. Manoranjini J
HEAD OF THE DEPARTMENT
Department of Computer Science
and Engineering
Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

Dr. Jinu Sophia J
COURSE FACULTY
Associate professor
Department of Computer Science
and Engineering
Rajalakshmi Engineering College

Submitted for the End Semester Examination held on: _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I, S BALAJI [2116240701069], hereby declare that the project report titled "City Pulse – Civic Issue Reporting and Tracking System" submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report — including its data model, forms, reports, pages, Deluge scripts, workflows and schedules — was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature (such as Zia or the AI application builder), code snippet or external resource used in this work has been duly acknowledged in the report.

Place: Chennai

Date: _____

SIGNATURE OF THE STUDENT
S BALAJI
[2116240701069]

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman Thiru. S. Meganathan, Chairperson Dr. (Mrs.) Thangam Meganathan and Vice-Chairman Mr. Abhay Meganathan for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal Dr. S.N. MURUGESAN, Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank Dr. Manoranjini J, Head of the Department, Department of Computer Science and Engineering for the guidance and support provided throughout the course.

I owe my sincere thanks to my course faculty, Dr. Jinu Sophia J, Associate professor, for the valuable guidance, reviews and feedback that shaped this application and report.

I also acknowledge Zoho Creator Education Program for providing access to the Zoho Creator platform and learning resources, and I thank my family and friends for their unwavering support.

S BALAJI

ABSTRACT

Cities receive numerous civic complaints, such as potholes, uncollected garbage, broken streetlights and water leakage, through calls, letters and online platforms. Managing these complaints manually often leads to duplicate reports, unclear department assignments, inaccurate locations and delays in resolution. The lack of centralised tracking also makes it difficult for citizens and officials to monitor progress and prioritise issues.

City Pulse is a centralised low-code application developed using Zoho Creator to manage civic complaints across city wards. It includes modules for users, wards, departments, problem reports, clusters, comments, confirmations, updates and notifications. Deluge scripting and workflows automate report clustering, department assignment, SLA tracking, notifications, escalation and auto-closure. The application also provides map views, reports, citizen and officer pages, and a customer portal with citizen, officer and moderator access profiles.

The application demonstrates an automated and transparent approach to civic issue management through centralised reporting, problem prioritisation and resolution tracking. Email and in-app notifications are implemented to keep users informed. The application includes role-based access controls and workflow automation. SMS, push notifications, live integrations and production deployment are identified as areas for further implementation and verification.

Keywords: Low-Code Development; Zoho Creator; Deluge Scripting; Workflow Automation; Role-Based Access Control; Civic Issue Reporting.

PROJECT AT A GLANCE

Attribute	Details
Project Title	City Pulse – Civic Issue Reporting and Tracking System
Application Domain	Smart city and municipal civic grievance management (city wards of Chennai)
Student Name & Register No.	K M BUSHRA BANU [2116240701084]
Development Platform	Zoho Creator low-code platform
Application Scope	User and ward administration, problem reporting with map location and photos, automatic clustering, confirmations, comments, follows, officer updates, resolutions, notifications, announcements, moderation flags, contribution scores, reports, map view and home page
Forms	15 forms (14 data-entry forms and 1 configuration form)
Automation	15 form workflows, 3 scheduled workflows and 1 reusable Deluge function (cp.notifyUser)
Process Control	Seven-state Problem Cluster lifecycle (Emerging, Active, Under Resolution, Resolved, Closed, Rejected, Reopened) driven by the Updates and Resolutions forms
Reports	16 reports: 14 list reports and 2 map views of Problem Clusters
Pages	6 pages: Home, Map View, Report a Problem, My Reports, Leaderboard and a widget page
Access Control	Customer portal with citizen, officer, moderator and default Customer profiles; staff role captured in the Users form
External Integration Points	Email notifications through Zoho Creator mail; map and geo-location through the Address field and map reports. SMS and push channels are listed as options and need separate configuration
Deployment Status	Application built (export version 1.0, generated 30-Oct-2026); published URL and production deployment evidence not supplied

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	5
	PROJECT AT A GLANCE	6
	LIST OF TABLES	10
	LIST OF FIGURES	11
	LIST OF ABBREVIATIONS	12
1	INTRODUCTION	13
1.1	Background	
1.2	Problem Statement	
1.3	Objective	
1.4	Scope of the Project	
1.5	Justification for a Low-Code Approach	
1.6	Organisation of the Report	
2	LITERATURE SURVEY AND SYSTEM STUDY	16
2.1	Study of the Existing System	
2.2	Limitations of the Existing System	
2.3	Low-Code versus Traditional Development	
2.4	Literature Survey	
2.5	Emerging Trends in Low-Code Development	
2.6	Proposed System	
3	REQUIREMENT ANALYSIS	19
3.1	Stakeholders and User Roles	
3.2	Functional Requirements	
3.3	Non-Functional Requirements	
3.4	Hardware and Software Requirements	
3.5	Use Case Model	

CHAPTER NO.	TITLE	PAGE NO.
4	SYSTEM DESIGN	22
4.1	System Architecture	
4.2	Data Model	
4.3	Form Design	
4.4	Process and Workflow Design	
4.5	User Interface and Navigation Design	
4.6	Access Control Design	
5	IMPLEMENTATION	27
5.1	Application Setup	
5.2	Forms and Data Capture	
5.3	Reports	
5.4	Pages and Dashboards	
5.5	Deluge Scripting	
5.6	Workflows and Automation	
6	TESTING AND DEBUGGING	36
6.1	Testing Strategy	
6.2	Test Cases	
6.3	Debugging Deluge Scripts	
6.4	Defect Log	
6.5	User Acceptance Testing	
7	DEPLOYMENT, SECURITY AND GOVERNANCE	39
7.1	Environments and Deployment Strategy	
7.2	Performance Optimisation	
7.3	Security Features	
7.4	Governance and Administration	
8	RESULTS AND DISCUSSION	41
8.1	Application Output	
8.2	Achievement of Objectives	
8.3	Comparison with the Existing System	
8.4	Limitations of the Low-Code Approach	
8.5	Reflection on the No-Code → Low-Code → Pro-Code Spectrum	

CHAPTER NO.	TITLE	PAGE NO.
9	CONCLUSION AND FUTURE ENHANCEMENTS	44
9.1	Conclusion	
9.2	Future Enhancements	
9.3	Learning Outcomes and Career Relevance	
10	REFERENCES	46
11	APPENDIX A – FEATURE COVERAGE CHECKLIST	47
12	APPENDIX B – REPORTS AND PAGES CATALOGUE	48
13	APPENDIX C – DELUGE LOGIC SUMMARY	50
14	APPENDIX D – USER MANUAL	52
15	APPENDIX E – EVALUATION RUBRIC	54
16	APPENDIX F – VIVA-VOCE RECORD SHEET	56

LIST OF TABLES

Table 1.1: Project Objectives and Measures
Table 2.1: Low-Code and Traditional Development Comparison
Table 2.2: Literature Survey Sources
Table 3.1: Stakeholders and User Roles
Table 3.2: Functional Requirements
Table 3.3: Non-Functional Requirements
Table 3.4: Hardware and Software Requirements
Table 3.5: Use Case Descriptions
Table 4.1: Entities and Relationships
Table 4.2: Form Design Groups
Table 4.3: Field Specification – Problem Reports
Table 4.4: Field Specification – Problem Clusters
Table 4.5: Access Control Design
Table 5.1: Forms Implemented
Table 5.2: Reports Implemented
Table 5.3: Pages Implemented
Table 5.4: Deluge Logic Summary
Table 5.5: Workflow Summary
Table 6.1: Test Cases
Table 6.2: Defect and Validation Log
Table 7.1: Deployment Checklist
Table 7.2: Security Controls Implemented
Table 8.1: Objective Achievement Mapping
Table 8.2: Comparison with the Existing System
Table A.1: Feature Coverage Checklist
Table B.1: Scheduled Monitoring
Table D.1: Troubleshooting
Table E.1: Evaluation Rubric for the Individual Project
Table E.2: Performance Level Descriptors
Table F.1: Viva-Voce Questions and Responses
Table F.2: Marks Summary

LIST OF FIGURES

Figure 2.1: City Pulse civic issue process represented as the proposed controlled workflow
Figure 3.1: Primary actors and the City Pulse application
Figure 4.1: Logical architecture of the City Pulse application
Figure 4.2: Conceptual relationships between core City Pulse entities
Figure 4.3: Problem cluster status lifecycle from first report to closure
Figure 5.1: City Pulse Application in Zoho Creator
Figure 5.2: Problem Reports Form in Live Mode
Figure 5.3: All Problem Clusters Report
Figure 5.4: Problem Clusters Map View
Figure 5.5: City Pulse Home Page
Figure 5.6: Report a Problem Page
Figure 5.7: Leaderboard Page
Figure 5.8: Report clustering logic
Figure 5.9: Severity scoring and priority assignment
Figure 5.10: Notification function (cp.notifyUser) in the Deluge editor
Figure 5.11: Workflow list of the City Pulse application
Figure 6.1: Deluge Workflow Execution History
Figure 7.1: Environments
Figure 8.1: Scenario Step 1 – Citizen Reports a Problem
Figure 8.2: Scenario Step 2 – Automatic Cluster Creation and Notification

LIST OF ABBREVIATIONS

Abbreviation	**Expansion**
API	Application Programming Interface
CRUD	Create, Read, Update and Delete
ER	Entity–Relationship
GPS	Global Positioning System
KPI	Key Performance Indicator
LCAP	Low-Code Application Platform
RBAC	Role-Based Access Control
SLA	Service Level Agreement
UAT	User Acceptance Testing
UI	User Interface
URL	Uniform Resource Locator

CHAPTER 1

INTRODUCTION

1.1 Background

Every city generates a steady flow of everyday problems: damaged roads, overflowing bins, non-functioning streetlights, water leakage, blocked storm-water drains and illegal dumping. Fixing these problems depends on the right department learning about them quickly, understanding exactly where they are, judging how serious they are and being held accountable for resolving them within a reasonable time. Citizens, on the other hand, want a simple way to report a problem, to see that it has been noticed, and to know when it has been fixed.

The project scenario describes City Pulse, a citizen-powered civic issue platform for the wards of a large city such as Chennai. The existing process relies on phone calls, written complaints, messaging groups and social media posts that reach different offices in different formats. The proposed City Pulse application brings reporting, location capture, de-duplication, assignment, status tracking, resolution proof and citizen feedback into a single centralised Zoho Creator application.

1.2 Problem Statement

Unstructured complaint handling makes it difficult to know how many distinct problems really exist, which of them are urgent, and who is responsible for each. Duplicate reports inflate workloads, imprecise locations delay field visits, and missing feedback reduces citizen trust and participation.

- The same pothole, garbage heap or drain blockage is reported by many people as separate complaints.
- Location is described in words, so officers cannot see problems on a map or identify the correct ward.
- Responsibility is unclear because reports are not mapped to a category, department and fix-by date.
- Problems that cross their fix-by date are not escalated automatically.
- Citizens receive no acknowledgement, progress update or confirmation of resolution.

- There is no incentive for citizens to report genuine problems or to confirm that a fix has actually happened.

1.3 Objectives

- Design linked data forms for users, wards, departments, categories, reports and problem clusters.
- Automatically group nearby reports of the same category into a single problem cluster.
- Assign ward, department, SLA due date, severity score and priority without manual effort.
- Track confirmations, comments, follows, officer updates and resolutions with notifications to the people concerned.
- Escalate overdue problems and auto-close resolved problems using scheduled workflows.
- Provide reports, a map view and a home page that give citizens and officials a live view of civic issues.

Objective ID	Objective	Evidence / Output
OBJ-01	Design linked data forms for users, wards, departments, categories, reports and problem clusters.	Fifteen linked forms with lookup fields and one subform
OBJ-02	Automatically group nearby reports of the same category into a single problem cluster.	Cluster Report workflow (Deluge)
OBJ-03	Assign ward, department, SLA due date, severity score and priority without manual effort.	Cluster Report workflow and Update Scores schedule
OBJ-04	Track confirmations, comments, follows, officer updates and resolutions with notifications.	Form workflows and cp.notifyUser function
OBJ-05	Escalate overdue problems and auto-close resolved problems.	SLA Check and Auto Close schedules
OBJ-06	Provide reports, a map view and a home page for live visibility.	Sixteen reports and six pages

Table 1.1: Project Objectives and Measures

1.4 Scope of the Project

The project scope includes user registration and portal access, ward, department and category master data, problem reporting with map location, photographs and video, automatic clustering, confirmations, comments, follows, officer updates, resolutions with before and after

photographs, in-app and email notifications, announcements, moderation flags, contribution scores, reports, a map view, a home page with counters and configurable system settings.

The following items appear in the data model but are not enforced by the scripts supplied in the application export: SMS and push notification delivery, enforcement of the daily report limit and spam-strike limit, and merging of duplicate clusters. They are recorded as configuration parameters or fields and are discussed under future enhancements. The scheduled workflows and the portal-user workflow are present in the export in an inactive state and must be activated in the target environment.

1.5 Justification for a Low-Code Approach

Zoho Creator provides visual form building, lookup relationships, reports, map views, pages, customer portals, workflow configuration and Deluge scripting within one platform. City Pulse is process-oriented: most requirements involve data capture, validation, counting, status changes and notifications rather than a completely custom user interface. Low-code development shortens repetitive implementation work and makes routine changes to forms, reports and workflows easier to maintain. Platform capabilities and limitations are discussed with reference to official Zoho Creator documentation [1].

1.6 Organisation of the Report

Chapter 2 studies the existing process and the proposed solution. Chapter 3 presents stakeholders and requirements. Chapter 4 describes architecture, data relationships and process design. Chapter 5 documents the application modules and automation. Chapter 6 presents the test approach and test cases. Chapter 7 discusses deployment, security and governance. Chapter 8 reviews the outcomes and limitations. Chapter 9 concludes the report and identifies future enhancements.

CHAPTER 2

LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

In the existing scenario, a citizen who notices a problem phones a helpline, writes to the ward office, or posts a photograph on social media. The complaint is noted manually, often without precise coordinates, and is forwarded to a department by phone or messaging. Several citizens may report the same problem independently and each complaint is treated as a separate item. Officers prioritise by instinct or by the loudest complaint, deadlines are not tracked consistently, and the citizen is rarely told when work starts or finishes.

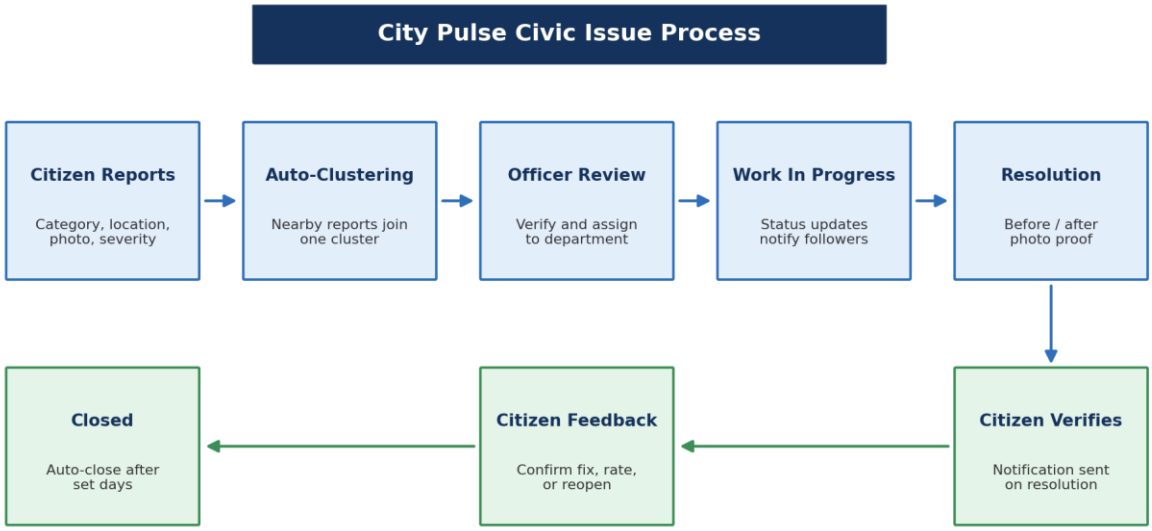


Figure 2.1: *City Pulse civic issue process represented as the proposed controlled workflow*

2.2 Limitations of the Existing System

- Fragmented complaints make it difficult to know how many distinct problems exist.
- Duplicate reports waste officer time and hide the true scale of a problem.
- Location details are informal, so the ward and department are identified late or incorrectly.
- Fix-by dates are not tracked, so overdue problems are not escalated automatically.

- Citizens do not receive acknowledgement, progress updates or proof of resolution.
- Management reports require manual consolidation and may be out of date.

2.3 Low-Code versus Traditional Development

Criterion	Traditional Pro-Code Development	Low-Code on Zoho Creator
Development effort	Requires a custom UI, database, APIs and deployment setup.	Uses visual form, report and page builders with configured workflows.
Data capture	Forms and validation are coded individually.	Forms, fields, lookups and subforms are configured visually.
Location features	Requires map libraries and geocoding integration.	Address field and map reports are provided by the platform.
Automation	Requires application logic and scheduled jobs to be implemented.	Workflow rules, schedules and Deluge scripts support event-driven automation.
Access for citizens	Requires custom authentication and role management.	Customer portal with configurable profiles is available.
Hosting	Infrastructure and release process are managed separately.	Platform hosting and application publishing are provided by the service.
Customisation	Offers broad control over architecture and UI.	Customisation is bounded by platform capabilities and plan limits.
Maintenance	Requires code-level maintenance and deployment discipline.	Routine changes may be simpler, but scripts and permissions still need testing.

Table 2.1: Low-Code and Traditional Development Comparison

2.4 Literature Survey

The following sources provide the platform and process background for this project. The supplied project brief does not include a formal research-paper survey; the table therefore focuses on authoritative platform documentation, established civic-reporting platforms and a security reference rather than claiming results from experiments that were not carried out.

S.No.	Source	Focus	Relevance
1	Zoho Creator documentation [1]	Forms, reports, pages and application building	Primary reference for platform capabilities.
2	Zoho Deluge documentation [2]	Scripting language and task syntax	Supports validation, clustering and automation logic.
3	Zoho Creator workflow documentation [3]	Form-event workflow rules	Relevant to report, confirmation, update and resolution triggers.
4	Zoho Creator schedules documentation [4]	Recurring and scheduled actions	Relevant to the daily scoring, SLA and auto-close jobs.
5	Zoho Creator customer portal documentation [5]	Portal users and profiles	Relevant to citizen, officer and moderator access.
6	FixMyStreet, mySociety [6]	Public problem reporting with map location	Reference model for citizen-to-authority reporting.

S.No.	Source	Focus	Relevance
7	SeeClickFix [7]	Civic issue reporting and tracking	Reference model for status updates and public visibility.
8	OWASP Application Security Verification Standard [8]	Application security controls	General checklist for access and data protection.

Table 2.2: Literature Survey Sources

2.5 Emerging Trends in Low-Code Development

Low-code platforms increasingly combine visual modelling, reusable components, event-driven automation, integrations and analytics. For City Pulse, these patterns support a single operational data model, repeatable status changes and map-based reporting. Automated checks should still be designed carefully: workflows need duplicate prevention, clear failure handling, role-appropriate permissions and testing with both valid and invalid records. Civic-technology platforms also show the value of public visibility and citizen participation, which City Pulse addresses through confirmations, follows, comments and contribution points.

2.6 Proposed System

The proposed system centralises civic problem information in linked Zoho Creator forms. Submitting a report triggers clustering, ward and department assignment and an SLA due date; confirmations and comments update counters and award points; officer updates and resolutions change the cluster status and notify the reporter and followers; a daily schedule computes severity and priority, escalates overdue problems and closes resolved problems after a configured period. Reports, a map view and the Home page provide visibility across the problem lifecycle.

CHAPTER 3

REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Role	Responsibilities
Citizen	Register, report problems with location and photos, confirm or comment on existing problems, follow problems of interest, track own reports, receive notifications and earn contribution points.
Moderator	Review new and flagged reports, comments and users; reject spam, duplicate or out-of-scope reports; keep community content appropriate.
Officer	Work on problems assigned to the department or ward, post status updates, submit resolutions with before and after photographs and respond to SLA alerts.
Admin	Maintain wards, departments, categories and system settings; manage users and access; monitor reports and the map.
Department Head	Receives SLA-breach escalation emails using the head email recorded in the Departments form.

Table 3.1: Stakeholders and User Roles

3.2 Functional Requirements

ID	Requirement	Role(s)	Priority
FR-01	Register users with a unique email, role, ward, notification preferences and language; require acceptance of terms.	Citizen / Admin	High
FR-02	Submit a problem report with category, description, address on map, severity, photos, optional video and an anonymous option.	Citizen	High
FR-03	Reject a report when no map location is chosen and capture latitude and longitude from the selected address.	System	High
FR-04	Link each report to the logged-in reporter, creating a user record automatically when none exists.	System	High
FR-05	Group a new report with an open problem cluster of the same category within the cluster radius; otherwise create a new cluster.	System	High
FR-06	Assign the nearest active ward, the category's department and an SLA due date to each new cluster.	System	High
FR-07	Allow citizens to confirm a problem as Still Exists, Worsened, Improved or No Longer Exists, and block a second confirmation by the same user on the same day.	Citizen / System	High
FR-08	Allow comments and follows, prevent duplicate follows and keep comment and follower counts up to date.	Citizen / System	Medium
FR-09	Record status updates against a cluster, validate the new status, apply it and notify the reporters and followers.	Officer / Moderator	High
FR-10	Record a resolution with description, before and after photographs and days taken; mark the cluster Resolved and notify reporters.	Officer	High
FR-11	Reopen a resolved or closed problem when citizens confirm that it still exists.	System	Medium

ID	Requirement	Role(s)	Priority
FR-12	Create in-app notifications and send email where the user has enabled email notifications.	System	High
FR-13	Compute severity score, priority and persistence days daily and promote Emerging clusters to Active at the threshold.	System	High
FR-14	Flag clusters that pass the SLA due date as overdue, raise the escalation level and email the department head and administrator.	System	High
FR-15	Automatically close problems that have remained Resolved for the configured number of days.	System	Medium
FR-16	Award contribution points for reports, confirmations and comments and display a leaderboard.	System / Citizen	Medium
FR-17	Allow flags on reports, comments and users, and publish announcements for wards.	Citizen / Moderator / Admin	Medium
FR-18	Provide reports, a map view, a My Reports page and a Home page with live counters.	All roles	Medium

Table 3.2: Functional Requirements

3.3 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Data integrity	Lookup relationships, unique fields and validation should keep linked records consistent.
NFR-02	Security and privacy	Users should access only the records and actions permitted by their portal profile; anonymous reporting should be supported.
NFR-03	Usability	Forms should be simple enough for a citizen to report a problem in a few steps on a mobile browser, with English and Tamil language options.
NFR-04	Maintainability	Thresholds, radius, points and SLA values should be configurable in the Settings and Category forms instead of being hard-coded.
NFR-05	Auditability	Status changes, resolutions and notifications should retain traceable records.
NFR-06	Availability	Application access depends on platform availability and network connectivity.
NFR-07	Performance	Reports and counters should filter data appropriately and avoid unnecessary repeated queries.

Table 3.3: Non-Functional Requirements

3.4 Hardware and Software Requirements

Category	Specification
Platform	Zoho Creator low-code application platform
Scripting	Deluge for form validation, workflow automation, scheduled logic and the notification function
Client	Desktop, laptop or smartphone with a modern browser and a stable internet connection; location access for map selection
Browser	Current version of Google Chrome, Microsoft Edge or Mozilla Firefox
Development	Zoho Creator form, report, page, workflow and schedule builders

Category	Specification
Optional integration tools	SMS and push notification providers; credentials and endpoint tests are required before live use

Table 3.4: Hardware and Software Requirements

3.5 Use Case Model

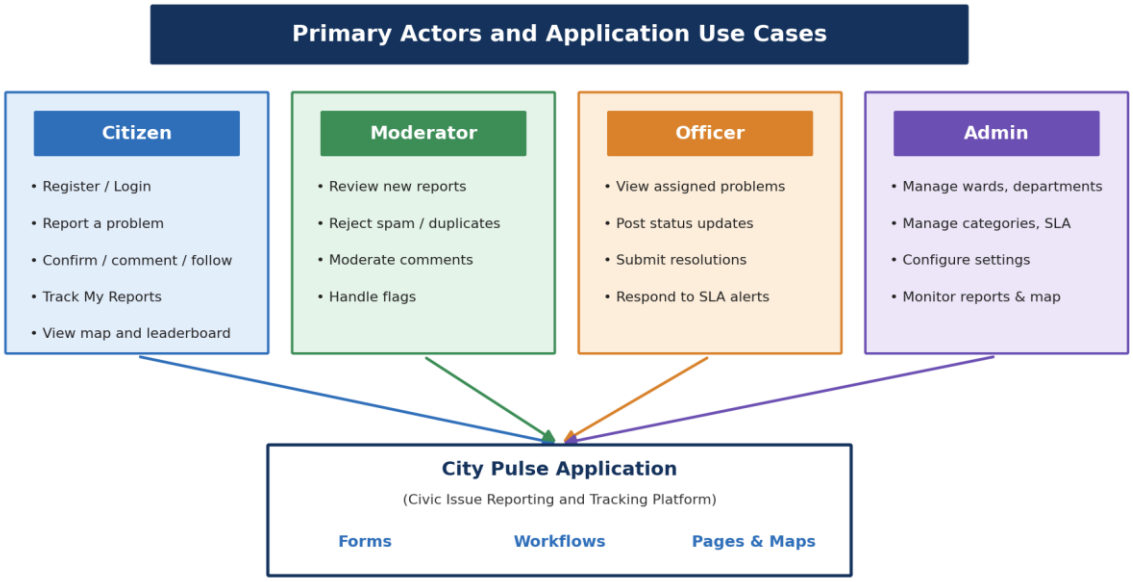


Figure 3.1: *Primary actors and the City Pulse application*

Use Case	Precondition	Main flow	Result
Submit report	Citizen is logged in to the portal.	Choose category, describe the problem, select location, set severity, attach photos and submit.	Report is recorded and linked to a new or existing cluster; citizen is notified.
Confirm problem	Cluster exists and citizen has not confirmed it today.	Choose a confirmation type, add comment or photo and submit.	Confirmation count and points increase; resolved problems may be reopened.
Update status	Officer or moderator has access to the cluster.	Select the cluster, choose a new status, add a note and submit.	Cluster status changes; reporters and followers are notified.
Resolve problem	Cluster is being worked on.	Enter resolution details with before and after photographs.	Cluster is marked Resolved; an update is logged and reporters are asked to verify.
Monitor SLA	Cluster has an SLA due date.	Daily schedule checks for clusters past the due date.	Cluster is marked overdue, escalated and an email is sent.

Table 3.5: Use Case Descriptions

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

The architecture separates the user-facing layer, the automation layer and the stored application records. Citizens and staff interact with Zoho Creator forms, reports, pages and the map. Deluge workflows, form-event validations and daily schedules enforce the process rules. Linked records provide the operational data used by reports, the map and the Home page counters.

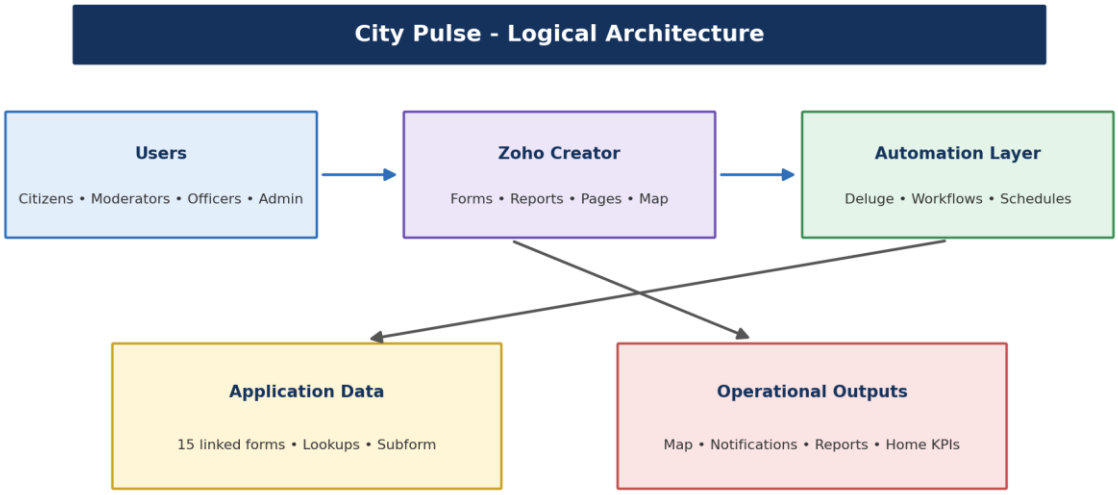


Figure 4.1: *Logical architecture of the City Pulse application*

4.2 Data Model

The data model uses master records for relatively stable information (Users, Wards, Departments, Problem Categories, Settings) and transaction records for civic activity. Lookup fields link transactions to master records. Problem Cluster is the central record: every Problem Report belongs to one cluster, and Confirmations, Comments, Follows, Updates, Resolutions and Notifications all point to a cluster. Counters stored on the cluster (reports, confirmations, comments and followers) avoid repeated counting in reports.

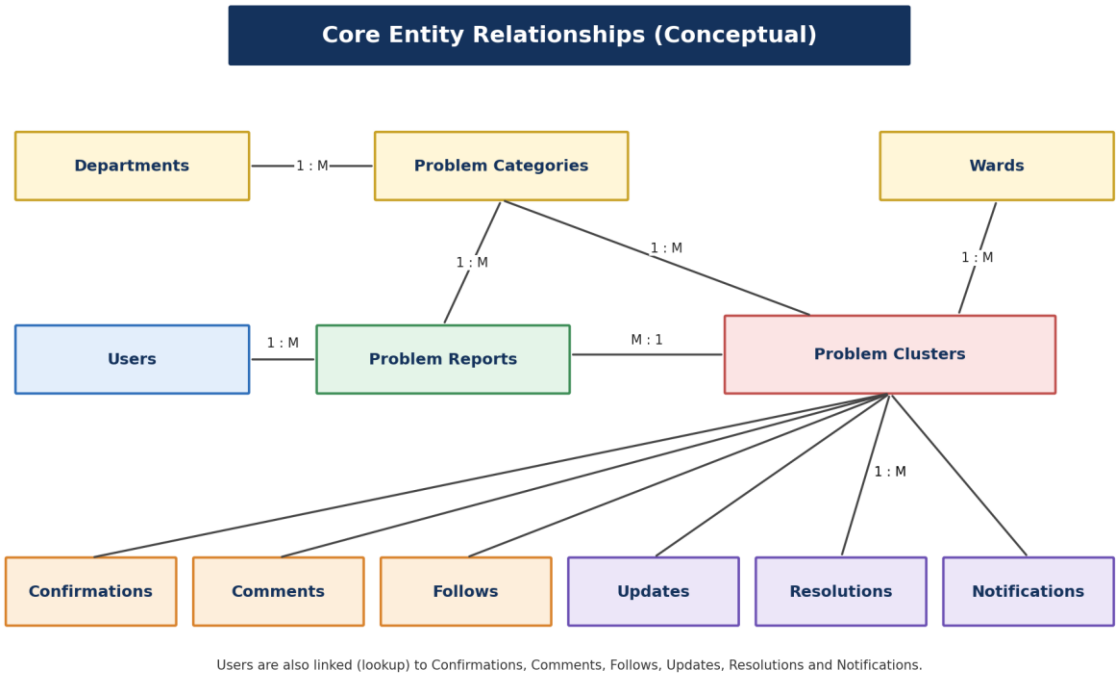


Figure 4.2: *Conceptual relationships between core City Pulse entities*

Entity	**Relationship / Purpose**
Users	Citizens, moderators, officers and admins; holds ward, department, notification preferences, contribution score and account status.
Wards / Departments	Geographic and organisational masters. A ward has a centre latitude and longitude; a department has a head name and email used for escalation.
Problem Categories	Defines severity weight, SLA days, cluster radius, responsible department and emergency flag. One category has many reports and clusters.
Problem Reports	Individual citizen submissions with location, photos, severity and status; each report is linked to one cluster.
Problem Clusters	One real-world problem grouping many reports; stores counts, severity score, priority, trend, SLA date and status.
Confirmations / Comments / Follows	Citizen participation records linked to a cluster and a user.
Updates / Resolutions	Officer actions: status updates and resolution proof linked to a cluster.
Notifications / Announcements	In-app messages for users and public notices for wards.
Flags / Settings	Moderation flags on reports, comments or users, and the single-row configuration of thresholds and points.

Table 4.1: Entities and Relationships

4.3 Form Design

Form group	Design considerations
Master forms	Users, Wards, Departments and Problem Categories store reusable reference data; unique constraints prevent duplicate names, codes and emails.
Citizen input forms	Problem Reports, Confirmations, Comments and Follows are kept short; the logged-in user is filled in automatically by workflows.
Officer action forms	Updates and Resolutions capture status changes and resolution proof with photographs.
System forms	Notifications and Settings are written by workflows and administrators rather than by citizens.
Community and moderation forms	Flags and Announcements support moderation and public notices.

Table 4.2: Form Design Groups

The principal fields of the two most important forms are specified below.

Field	Type	Rule / Notes
Report_ID	Auto number	Unique report identifier.
Reporter	Lookup (Users)	Filled automatically from the logged-in user.
Category	Lookup (Problem Categories)	Mandatory.
Description	Multi-line text	Mandatory.
Address	Address (map)	Mandatory; latitude and longitude are read from it.
Latitude, Longitude	Decimal	Set by the Set Coordinates workflow.
Severity	Drop-down	Mandatory: Low, Medium, High, Critical.
Main_Photo / SubForm	Image / Subform	Main photo plus a subform of photos with captions.
Video	File upload	Optional.
Anonymus	Checkbox	Hides the reporter identity when selected.
Status	Drop-down	Draft, New, Under Review, Verified, Rejected.
Rejection_Reason	Drop-down	Spam, Duplicate, Fake, Out of Scope, Insufficient Details, Other.
Cluster, Wards	Lookup	Set by the Cluster Report workflow.
Duplicate_Flag, Spam_Flag	Checkbox	Moderation markers.

Table 4.3: Field Specification – Problem Reports

Field	Type	Rule / Notes
Problem_Title	Text	Generated as “category near ward”.
Category, Ward, Department	Lookup	Category mandatory; ward and department assigned automatically.
Latitude, Longitude	Decimal	Mandatory; used for clustering and the map.

Field	Type	Rule / Notes
Report_count, Confirmation_Count, Comment_Count, Follower_Count	Number	Counters, default 0.
First_Reported, Last_Reported	Date-time	Used for persistence calculation.
Persistence_Days, Severity_Score	Number / Decimal	Computed daily by the Update Scores schedule.
Priority	Drop-down	Low, Medium, High, Critical.
Trend	Drop-down	Rising, Stable, Falling.
SLA_Due_Date, Overdue, Escalation_Level	Date / Checkbox / Number	Maintained by the SLA Check schedule.
Verified_By, Verified_On, Resolved_On	Lookup / Date-time	Set when status becomes Active or Resolved.
Reopen_Count, Average_Rating, Merged_Into	Number / Decimal / Lookup	Reopen tracking, rating and merge support.
Status	Drop-down	Mandatory: Emerging, Active, Under Resolution, Resolved, Closed, Rejected, Reopened.

Table 4.4: Field Specification – Problem Clusters

4.4 Process and Workflow Design

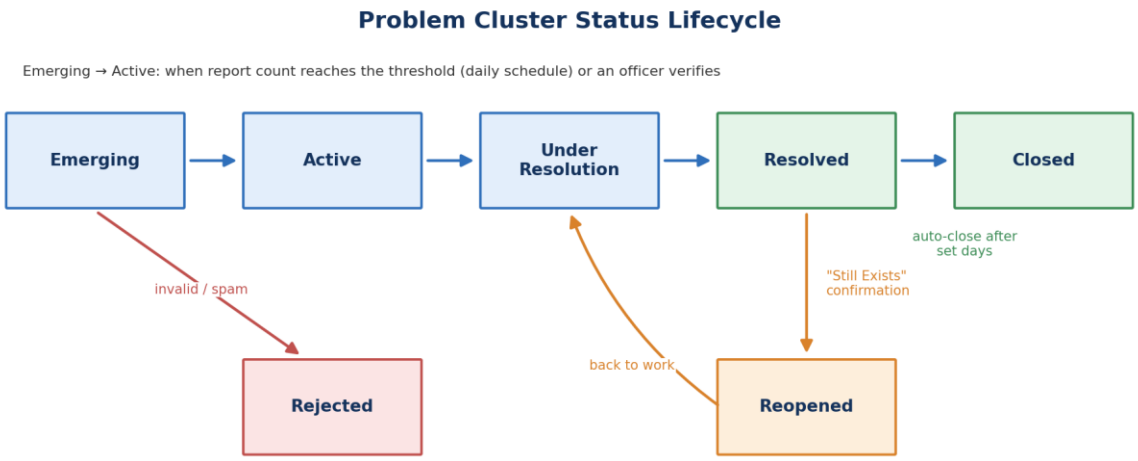


Figure 4.3: *Problem cluster status lifecycle from first report to closure*

The lifecycle of a problem is controlled through the Status field of the Problem Cluster. A new cluster starts as Emerging. It becomes Active when the number of reports reaches the Emerging threshold (checked by the daily scoring schedule) or when an officer verifies it through an update. Officers move it to Under Resolution and then Resolved; a resolution record or a status update to Resolved stores the resolution time. A daily schedule closes problems that have stayed

Resolved for the configured number of days. A confirmation of "Still Exists" on a resolved or closed problem reopens it. Invalid problems are set to Rejected. The Updates form validates that the new status is one of the seven allowed values, so illegal states cannot be entered. Zoho Creator Blueprint was not used in this application; the lifecycle is enforced through form validation and workflows.

4.5 User Interface and Navigation Design

The application is organised around role-specific tasks. Citizens need a simple route to report a problem, see problems on a map, view their own reports and notifications, and confirm, comment or follow. The Home page therefore offers three large buttons (Report a Problem, View Problems and My Notifications), two counters (open and resolved problems) and a list of problem clusters. Officers and moderators use the Problem Clusters, Updates, Resolutions and Flags reports. Administrators use the master-data and Settings reports. Status values and colours are kept consistent across pages.

4.6 Access Control Design

Role	Typical permitted actions	Restricted / controlled actions
Citizen	Create reports, confirmations, comments, follows and flags; view problem clusters, categories, wards, announcements and own notifications; use Map View, Report a Problem and My Reports pages.	Cannot post officer updates or resolutions, edit settings or edit other users' reports.
Moderator	Review reports, comments and flags; edit report status and rejection reason.	Cannot change system settings or master data unless permitted.
Officer	View assigned problems; post updates and resolutions; receive SLA alerts.	Cannot alter settings or other departments' data unless permitted.
Admin	Manage wards, departments, categories, settings and user accounts.	Privileged activity should be limited and reviewed.

Table 4.5: Access Control Design

CHAPTER 5

IMPLEMENTATION

5.1 Application Setup

The application “City Pulse” is built on Zoho Creator using visual builders for forms, reports, pages and workflows. It uses the date format dd-MMM-yyyy, the Asia/Kolkata time zone and a 24-hour time format, and is identified by the “CP” application icon. A consistent naming scheme is used for forms and fields because Deluge scripts refer to field link names. Before publishing changes, the developer verifies field link names, lookup targets, mandatory fields and workflow execution order.



Figure 5.1: *City Pulse Application in Zoho Creator*

5.2 Forms and Data Capture

No.	Form	Purpose and principal fields
1	Users	Full name, unique email, phone, role (Citizen, Moderator, Officer, Admin), ward, department, contribution score, reports count, spam strikes, account status, notification preferences, language and terms accepted.
2	Wards	Ward name (unique), ward number, zone, centre latitude and longitude, active flag.
3	Departments	Department name and code (unique), email, phone, head name and email, address, active flag.
4	Problem Categories	Category name (unique), description, icon, map colour, severity weight, department, SLA days, cluster radius, emergency flag and display order.
5	Problem Reports	Reporter, category, description, address, latitude and longitude, ward, severity, main photo, photo subform, video, anonymous flag, status, rejection reason and cluster.

No.	Form	Purpose and principal fields
6	Problem Clusters	Title, category, address, coordinates, ward, department, assigned officer, counters, severity score, priority, trend, SLA date, overdue flag, escalation level and status.
7	Confirmations	User, cluster, confirmation type, confirmed-on date, comment and photo.
8	Comments	Cluster, user, comment text, image, parent comment, posted-on time and visibility status.
9	Follows	User, cluster, followed-on time and active flag.
10	Updates	Cluster, updated-by, update type, previous and new status, comment, evidence photo and public visibility.
11	Resolutions	Cluster, resolved-by, resolution date, description, before and after photos, days taken, cost, status, citizen verification, rating and feedback.
12	Notifications	User, cluster, notification type (12 types), message, channel, created-on time and read status.
13	Flags	Flag type (report, comment or user), target, reported-by, reason, details, status and reviewer.
14	Announcements	Title, message, type, ward, image, start and end dates and published flag.
15	Settings	App name, cluster radius, emerging threshold, auto-close days, escalation days, points per action, daily report limit, spam strike limit, image size limit and admin and support emails.

Table 5.1: Forms Implemented

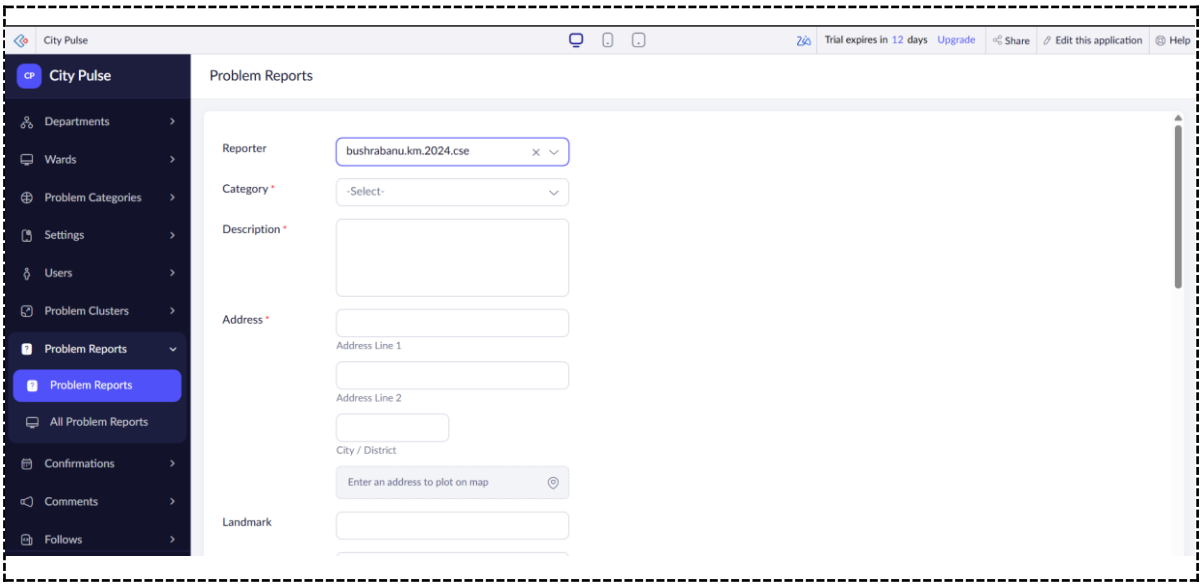


Figure 5.2: *Problem Reports Form in Live Mode*

5.3 Reports

Fourteen list reports provide a tabular view of each form, and two map reports plot problem clusters geographically using the Address field. The reports are listed below.

No.	Report	Use
1	All Departments	Review departments, heads and contact details.
2	All Wards	Review wards, zones and centre coordinates.
3	All Problem Categories	Review categories, SLA days, weights and radii.

No.	Report	Use
4	All Settings	Review thresholds, points and limits.
5	All Users	Review users, roles, wards, scores and account status.
6	All Problem Clusters	Review problems with counts, priority, SLA and status.
7	All Problem Reports	Review individual citizen reports and their status.
8	All Confirmations	Review confirmations of problems.
9	All Comments	Review comments and their visibility status.
10	Follows Report	Review followers of problems.
11	All Updates	Review the status-change history.
12	All Resolutions	Review resolutions, cost and days taken.
13	All Notifications	Review notifications and read status.
14	All Announcements	Review ward announcements.
15	Flags Report	Review moderation flags.
16	Problem Clusters Map	View problem clusters on a map.

Table 5.2: Reports Implemented

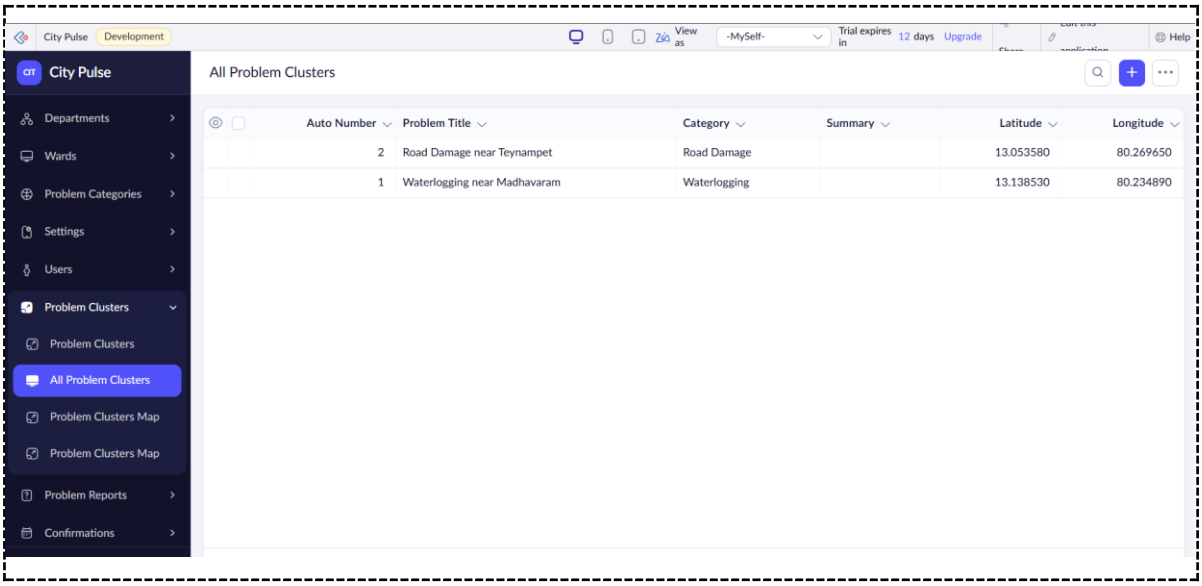


Figure 5.3: *All Problem Clusters Report*

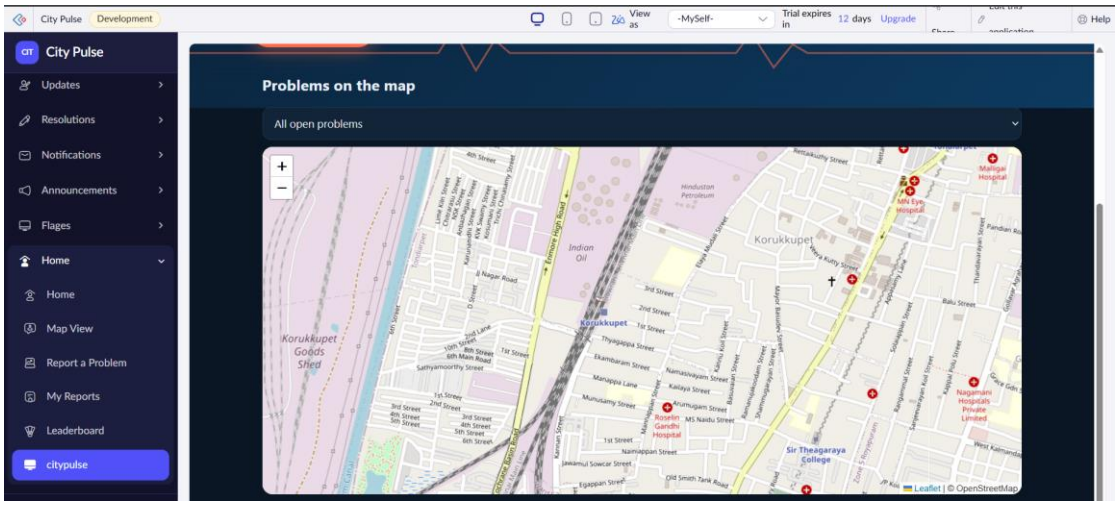


Figure 5.4: *Problem Clusters Map View*

5.4 Pages and Dashboards

Six pages provide the citizen-facing experience. The Home page opens with a welcome banner and three buttons — Report a Problem, View Problems and My Notifications. Below them, two live counters show the number of open civic issues (clusters not Resolved or Closed) and the number of issues resolved (Resolved or Closed), followed by the Problem Clusters report. The Map View page embeds the problem cluster map. The Report a Problem page embeds the Problem Reports form with a three-step guide (select category, pin location, attach proof). The My Reports page lists only the reports whose reporter email matches the logged-in user, with add, edit, delete, print and export disabled. The Leaderboard page presents a champion banner and a podium of top contributors.

No.	Page	Content
1	Home	Welcome banner, three action buttons, open and resolved problem counters and the Problem Clusters report.
2	Map View	Heading banner and the Problem Clusters map.
3	Report a Problem	Three-step guidance banner and the Problem Reports form.
4	My Reports	Status legend and a report filtered to the logged-in reporter; editing and export disabled.
5	Leaderboard	Champion banner and podium for the top three contributors (sample content; to be linked to contribution scores).
6	City Pulse (widget page)	Widget-based page imported from a template.

Table 5.3: Pages Implemented

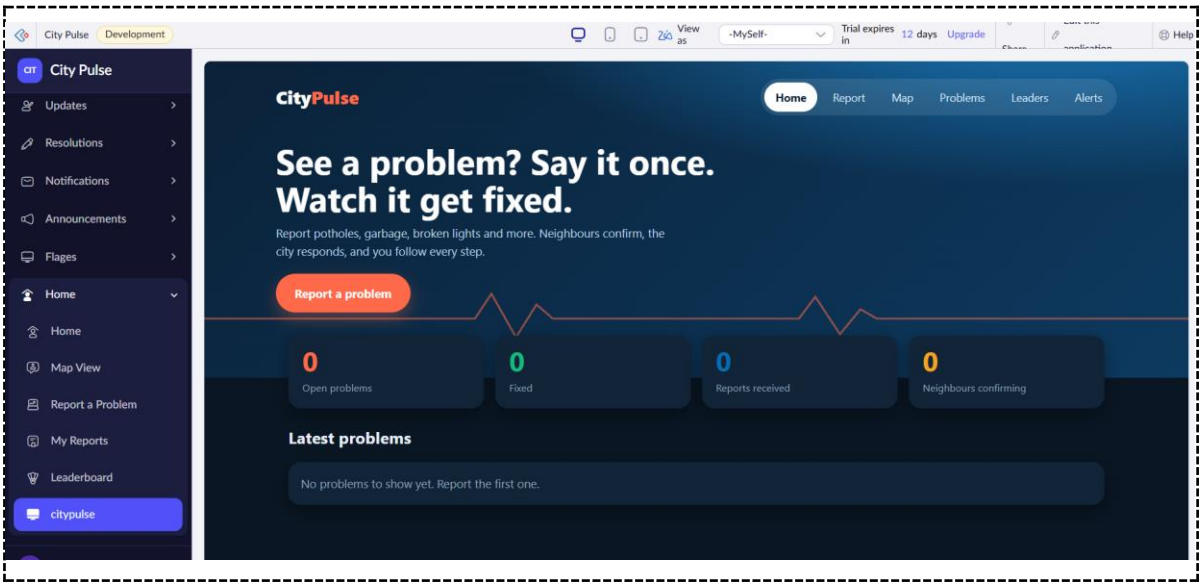


Figure 5.5: *City Pulse Home Page*

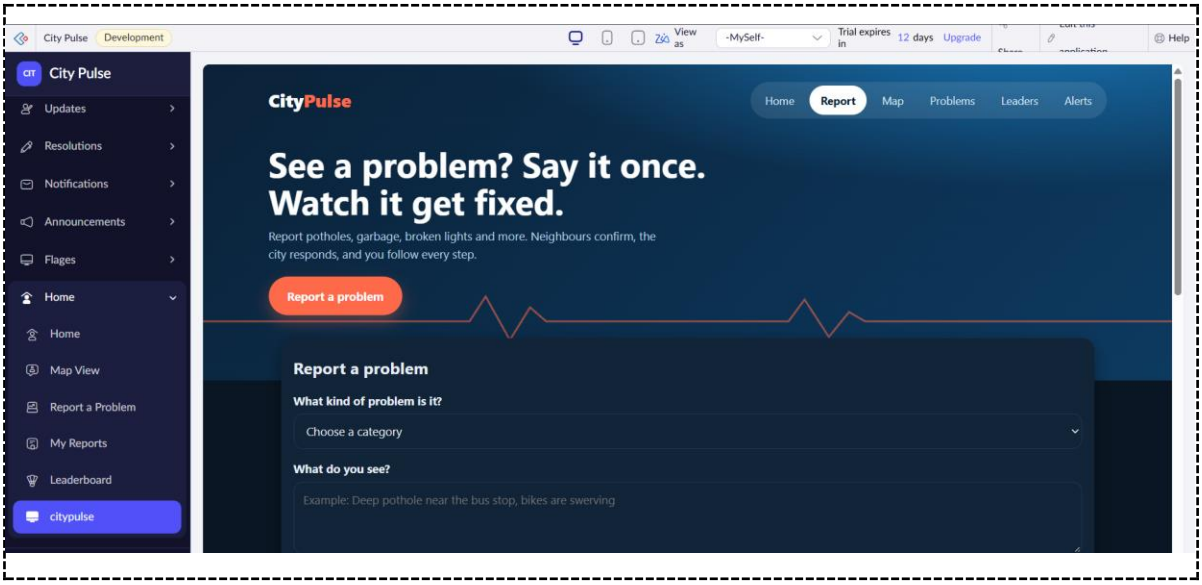


Figure 5.6: *Report a Problem Page*

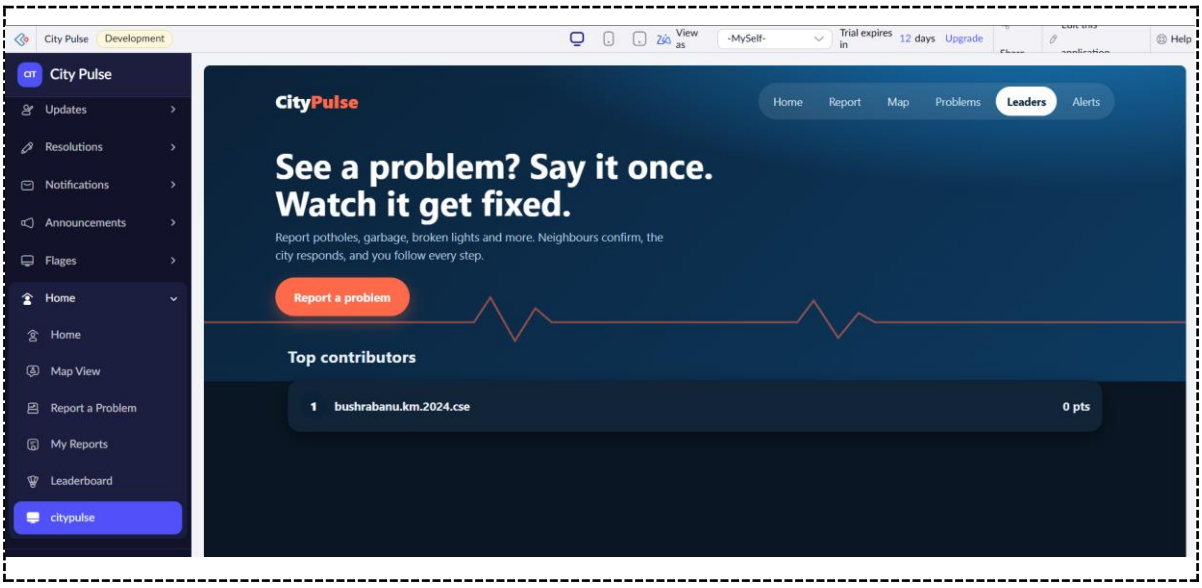


Figure 5.7: *Leaderboard Page*

5.5 Deluge Scripting

Deluge is used to implement field validation, geo-clustering, counters, scoring and notifications. Scripts are attached to the correct form event (on load, on validate or on success), use the actual field link names, and guard against empty lookups and repeated execution. The principal scripts are summarised in Table 5.4.

No.	Trigger / Area	Logic summary
1	Problem Reports – on load (Set Reporter)	Find the Users record matching the logged-in email; create a Citizen user when none exists; fill the Reporter field.
2	Problem Reports – on validate (Set Coordinates)	Read latitude and longitude from the Address field; show an alert and cancel submit when no map location is chosen.
3	Problem Reports – on success (Cluster Report)	Read settings and category; find the nearest active ward; join an open cluster of the same category within the radius or create a new one; link report and ward; award points; notify the reporter.
4	Confirmations – on validate (Block Duplicate)	Reject a second confirmation of the same cluster by the same user on the same day.
5	Confirmations – on success (Update Counts)	Increase the confirmation count; reopen resolved or closed problems when the type is “Still Exists”; award points.
6	Comments and Follows	Fill the user, prevent duplicate follows, and update comment and follower counters and points.
7	Updates – validate and success	Fill updated-by and previous status; reject an invalid new status; apply the status to the cluster; set verified and resolved details; notify reporters and followers.
8	Resolutions – validate and success	Fill resolved-by and days taken; mark the cluster Resolved; insert a public update; notify reporters to verify the fix.
9	Function cp.notifyUser	Insert an in-app notification and send an email when the user has not disabled email notifications.

No.	Trigger / Area	Logic summary
10	Schedules (Update Scores, SLA Check, Auto Close)	Compute severity and priority; flag and escalate overdue problems; close problems resolved for longer than the configured days.

Table 5.4: Deluge Logic Summary

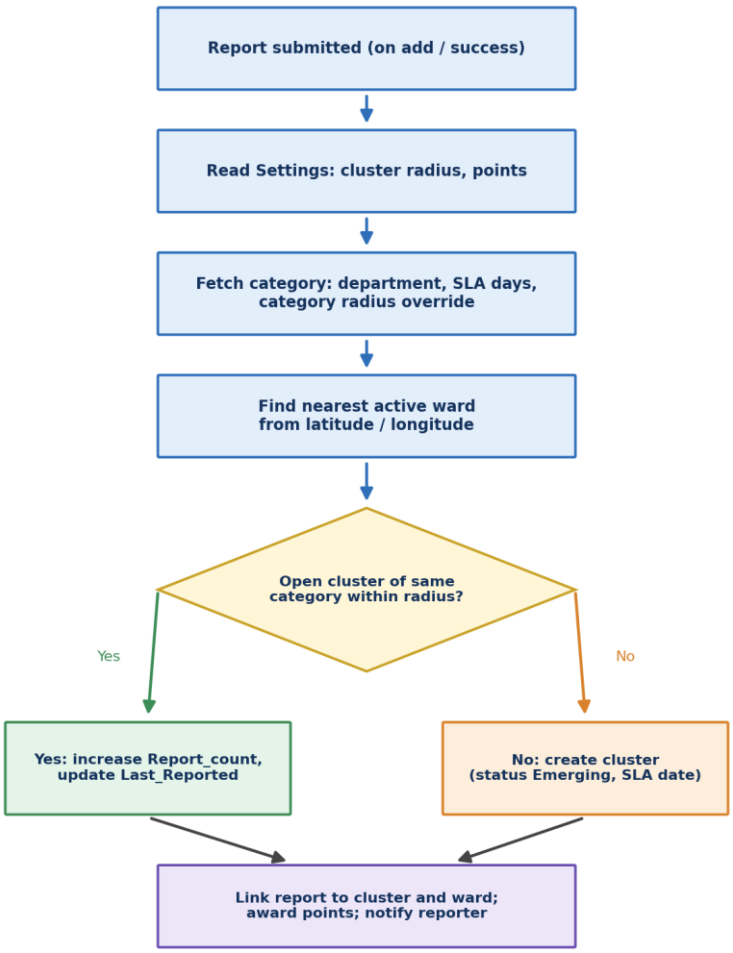


Figure 5.8: *Report clustering logic*

The severity score of an open cluster is calculated as the category severity weight multiplied by the number of reports, by the number of confirmations plus one, and by the number of days the problem has been open plus one. A score of 200 or more is Critical, 60 or more is High, 20 or more is Medium and anything lower is Low. Clusters that are still Emerging become Active when their report count reaches the configured threshold.

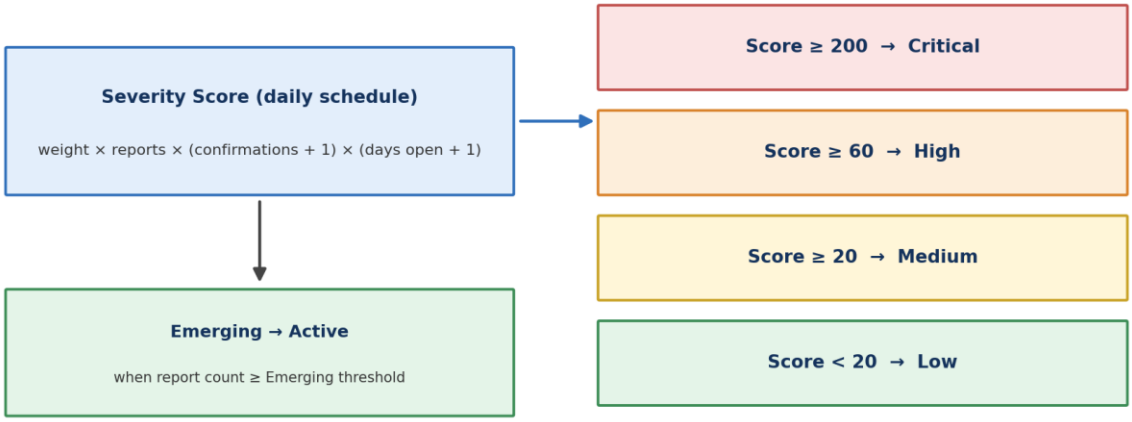


Figure 5.9: *Severity scoring and priority assignment*

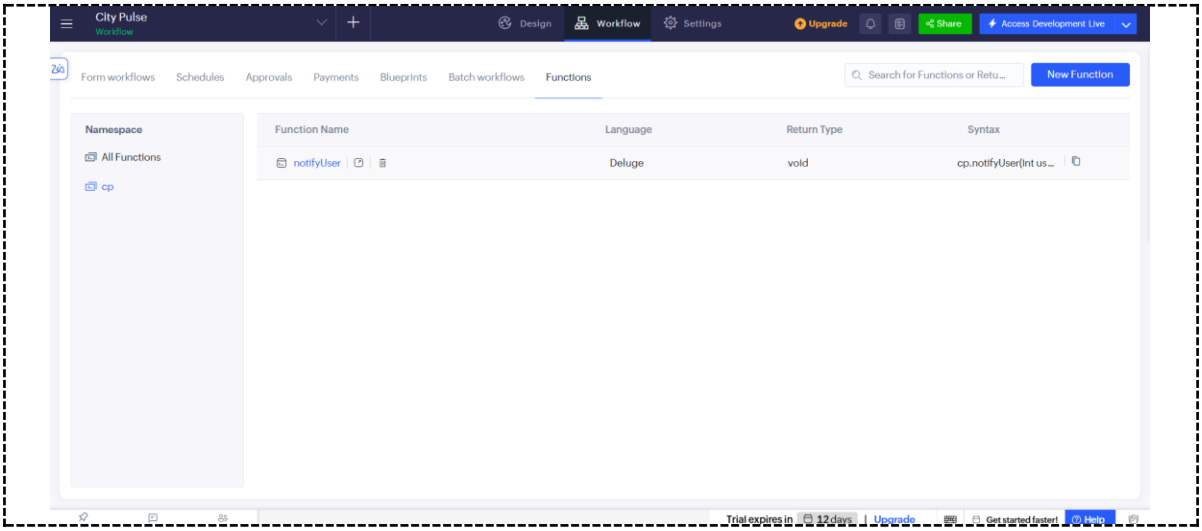


Figure 5.10: *Notification function (cp.notifyUser) in the Deluge editor*

5.6 Workflows and Automation

The application contains fifteen form workflows and three scheduled workflows, summarised in Table 5.5. In the application export used for this report the three schedules and the Add Portal User workflow have the status “inactive”; they must be activated in the target environment before production use.

No.	Workflow	Form / Type	Event	Status in export
1	Add Portal User	Users	On add – success	Inactive
2	Set Reporter	Problem Reports	On add – on load	Active
3	Set Coordinates	Problem Reports	On add – on validate	Active

No.	Workflow	Form / Type	Event	Status in export
4	Cluster Report	Problem Reports	On add – success	Active
5	Set Confirmation User	Confirmations	On add – on load	Active
6	Block Duplicate	Confirmations	On add – on validate	Active
7	Update Counts	Confirmations	On add – success	Active
8	Prepare Comment	Comments	On add – on load	Active
9	Update Comment Counts	Comments	On add – success	Active
10	Prepare Follow	Follows	On add – on validate	Active
11	Update Follower Count	Follows	On add – success	Active
12	Prepare Update	Updates	On add – on validate	Active
13	Apply Update	Updates	On add – success	Active
14	Prepare Resolution	Resolutions	On add – on validate	Active
15	Apply Resolution	Resolutions	On add – success	Active
16	Update Scores	Schedule	Daily at 06:00 (Asia/Kolkata)	Inactive
17	SLA Check	Schedule	Daily at 06:00 (Asia/Kolkata)	Inactive
18	Auto Close	Schedule	Daily at 06:00 (Asia/Kolkata)	Inactive

Table 5.5: Workflow Summary

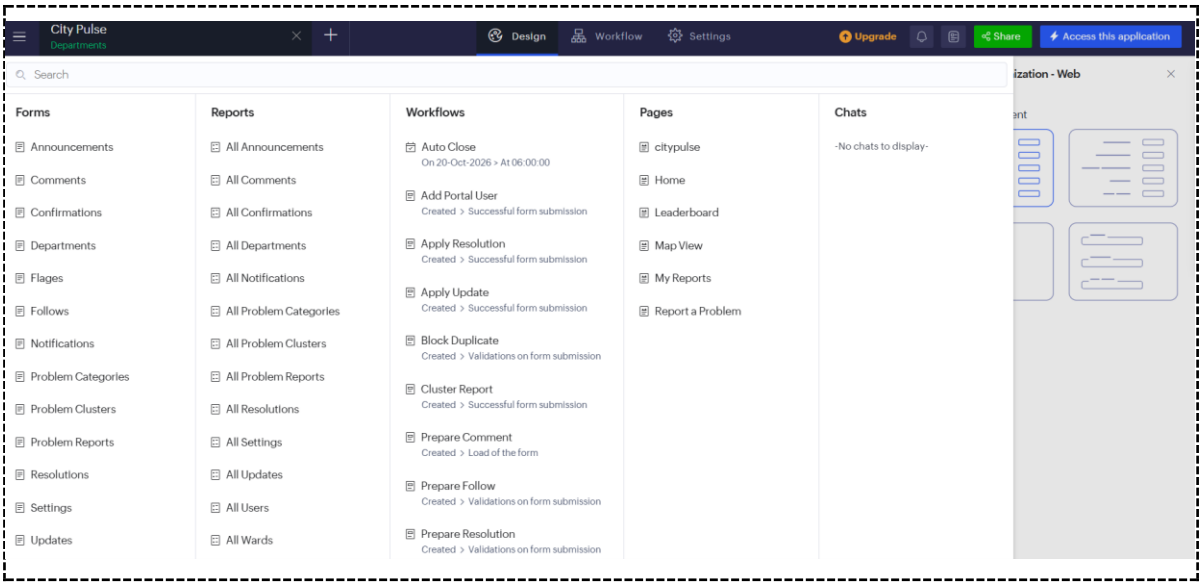


Figure 5.11: *Workflow list of the City Pulse application*

CHAPTER 6

TESTING AND DEBUGGING

6.1 Testing Strategy

Testing should cover individual field rules, workflow triggers, linked-record consistency and end-to-end citizen scenarios. Each test should be performed with valid and invalid data and the result recorded. The cases below are a practical test plan based on the implemented workflows; they should be marked Pass only after execution in the live application.

6.2 Test Cases

Test ID	Area	Test input / action	Expected result	Status
TC-01	Location validation	Submit a report without choosing a map location.	Submission is cancelled with an alert asking for a location.	To be executed
TC-02	First report	Submit a report where no open cluster exists nearby.	A new cluster is created with status Emerging, ward, department and SLA date.	To be executed
TC-03	Clustering	Submit a second report of the same category within the radius.	Report joins the same cluster and Report_count becomes 2.	To be executed
TC-04	Different category	Submit a nearby report of a different category.	A separate cluster is created.	To be executed
TC-05	Reporter points	Submit a report as a logged-in citizen.	Contribution score and reports count increase; a Report Received notification is created.	To be executed
TC-06	Duplicate confirmation	Confirm the same problem twice on one day.	Second confirmation is blocked with an alert.	To be executed
TC-07	Duplicate follow	Follow the same problem twice.	Second follow is blocked with an alert.	To be executed
TC-08	Status update	Post an update with status Active, then Under Resolution.	Cluster status changes; reporters and followers are notified.	To be executed
TC-09	Invalid status	Post an update with a status outside the allowed list.	Submission is rejected with an alert.	To be executed
TC-10	Resolution	Submit a resolution with after photo.	Cluster becomes Resolved; Days_Taken and a public update are recorded.	To be executed
TC-11	Reopen	Confirm “Still Exists” on a resolved problem.	Cluster status becomes Reopened and Reopen_Count increases.	To be executed
TC-12	Severity and priority	Run the Update Scores schedule.	Severity score, persistence days and priority are updated.	To be executed
TC-13	SLA breach	Run SLA Check for a cluster past its due date.	Cluster is marked overdue, escalation level rises and an email is sent once.	To be executed
TC-14	Auto close	Run Auto Close for a cluster resolved longer than the limit.	Cluster status becomes Closed.	To be executed
TC-15	Access control	Try to post an update or edit settings as a citizen.	Action is not available to the citizen profile.	To be executed

Table 6.1: Test Cases

6.3 Debugging Deluge Scripts

When a workflow fails, first confirm the form event and execution timing. Verify field link names, lookup values and null handling. Use temporary diagnostic messages or logs during development, then remove sensitive debug output before deployment. Check whether another workflow updates the same field and whether the same event can execute more than once. Test counter updates, notification creation and scheduled actions for repeated execution to prevent duplicate records or repeated emails. The execution history of each workflow shows the trigger, record, status and output of every run and is the first place to look when a script does not behave as expected.

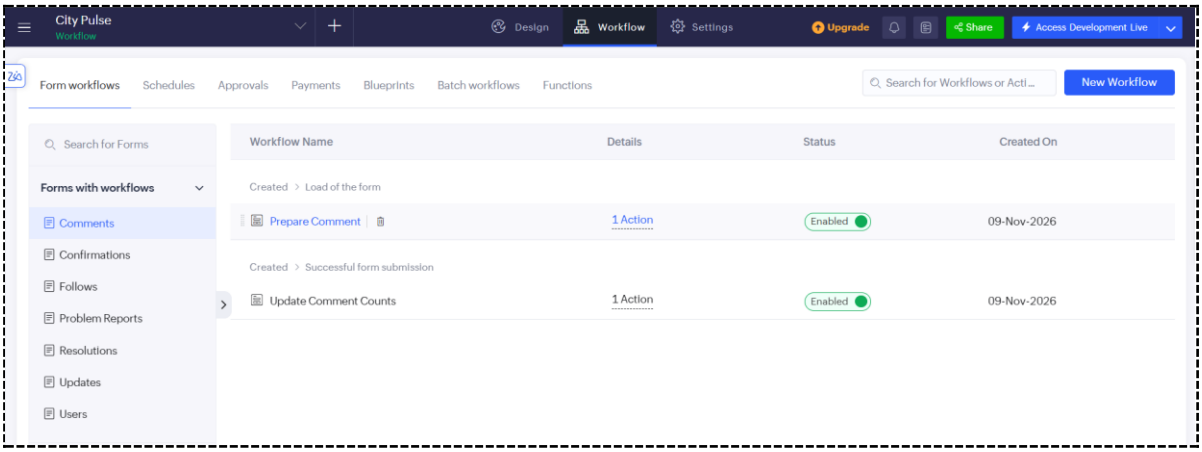


Figure 6.1: *Deluge Workflow Execution History*

6.4 Defect Log

The following potential defects were identified while reviewing the scripts and configuration and are recorded for verification in the live application.

Defect ID	Potential defect to monitor	Resolution / control	Status
D-01	In Update Counts, the reopen condition combines && and || without brackets, so any confirmation on a Closed problem may reopen it.	Bracket the status test, for example Still Exists AND (Resolved OR Closed).	Verify in application
D-02	Flag status list contains “Action” and “Taken” as two separate values.	Replace with the single value “Action Taken”.	Verify in application
D-03	Daily report limit and spam strike limit exist in Settings but no script enforces them.	Add validation on Problem Reports and increment Spam_Strikes on rejection.	Verify in application
D-04	Schedules and Add Portal User workflow are inactive in the export.	Activate and test them in the target environment.	Verify in application
D-05	Leaderboard page shows sample points rather than live scores.	Bind the podium to Users.Contribution_Score.	Verify in application

Defect ID	Potential defect to monitor	Resolution / control	Status
D-06	SLA Check could email repeatedly.	Overdue flag is set on first breach and excluded from later runs; test repeated runs.	Verify in application

Table 6.2: Defect and Validation Log

6.5 User Acceptance Testing

Acceptance testing should involve a citizen, a moderator and an officer following a complete scenario: register a citizen, submit a report, submit a second nearby report and check that both belong to one cluster, confirm and follow the problem, post officer updates, submit a resolution, verify notifications and the resolved counter on the Home page, and confirm that the problem closes automatically after the configured period. Acceptance should be based on traceable records, correct counters, appropriate access and clear status changes. No signed acceptance sheet was supplied with the application export.

CHAPTER 7

DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

The application should be developed and tested in a controlled development environment, promoted to a stage environment and then published to production. A release checklist should confirm that forms, lookup relationships, Deluge scripts, workflows, schedules, reports, pages, portal profiles and notification settings are configured in the target environment. The supplied export does not include a published portal URL or deployment record, so the final production deployment remains to be verified.

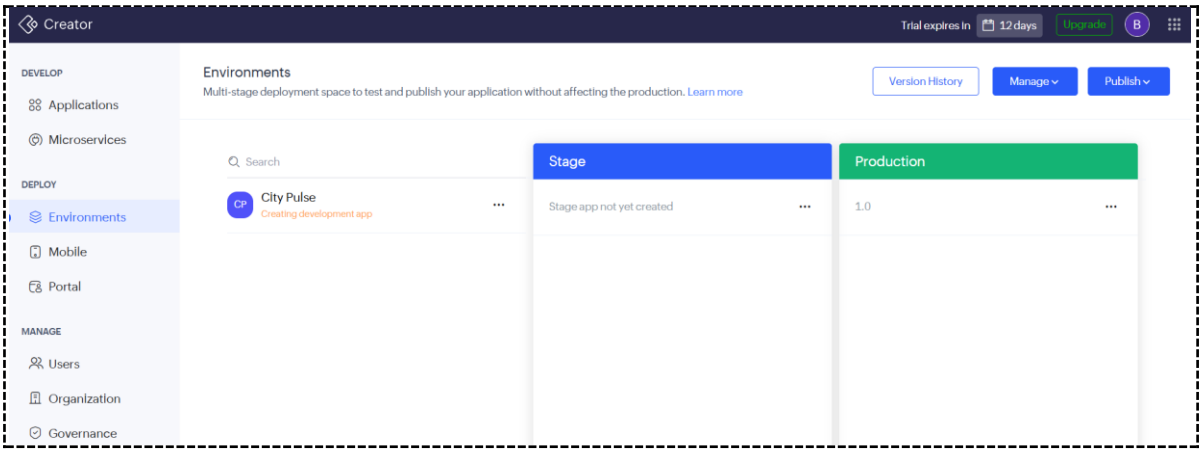


Figure 7.1: *Environments*

Deployment check	**Verification**
Data model	Confirm all forms, field link names, lookups and the subform.
Master data	Load wards with centre coordinates, departments with head emails, categories with SLA days and the Settings record.
Automation	Activate and test workflows and schedules with sample records.
Notifications	Verify sender settings, recipient addresses and delivery outcomes.
Portal	Test the citizen, officer and moderator profiles and the Add Portal User workflow.
Access	Test each role with least-privilege permissions.
Recovery	Define export, backup, recovery and change-control procedures.
Release	Record date, version, approver and known limitations.

Table 7.1: Deployment Checklist

7.2 Performance Optimisation

- Store counters (reports, confirmations, comments, followers) on the cluster instead of recounting in every report.
- Use report filters to limit data to relevant wards, statuses or dates.
- Limit clustering queries to a small bounding box around the new report.
- Exclude Resolved, Closed and Rejected clusters from scheduled scans where possible.
- Keep scheduled jobs daily and avoid repeated updates or repeated notifications.

7.3 Security Features

Control	**Application of the control**
Authentication	Citizens and staff sign in through the Zoho Creator customer portal.
Authorisation	Portal profiles (citizen, officer, moderator, Customer) limit forms, reports and pages; the Users form holds the staff role.
Privacy	Anonymous reporting option; users can opt in or out of email, push and SMS notifications.
Validation	Mandatory fields, unique emails and names, map location check, duplicate confirmation and follow checks, and allowed-status check.
Auditability	Status changes are retained in Updates; resolutions, flags and notifications keep their own records.
Abuse control	Flags, spam and duplicate markers, spam strikes and account status (Active, Suspended, Blocked); limits to be enforced by script.
Error handling	The Add Portal User workflow uses try and catch and emails the administrator; sensitive data should not be exposed in errors or logs.

Table 7.2: Security Controls Implemented

7.4 Governance and Administration

Governance covers who may change configuration, how changes are tested and how the application is maintained. Administrators should keep the Settings form as the single place for tunable values (radius, thresholds, points, limits), review portal profile permissions periodically, record each release with its date and version, and keep a short log of changes to workflows and schedules. Reviews of flags and rejected reports help keep community content reliable.

CHAPTER 8

RESULTS AND DISCUSSION

8.1 Application Output

The application structure covers the civic problem lifecycle from user registration through reporting, clustering, community confirmation, officer action, resolution, citizen verification and automatic closure. The following scenario illustrates the expected flow; screenshots of the live application should be inserted at the placeholders.

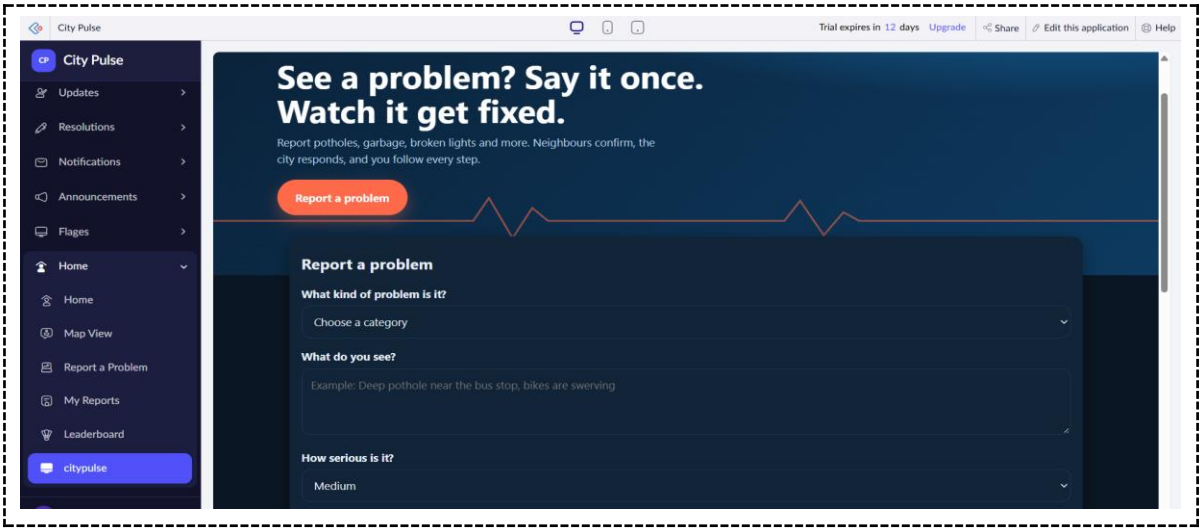


Figure 8.1: *Scenario Step 1 – Citizen Reports a Problem*

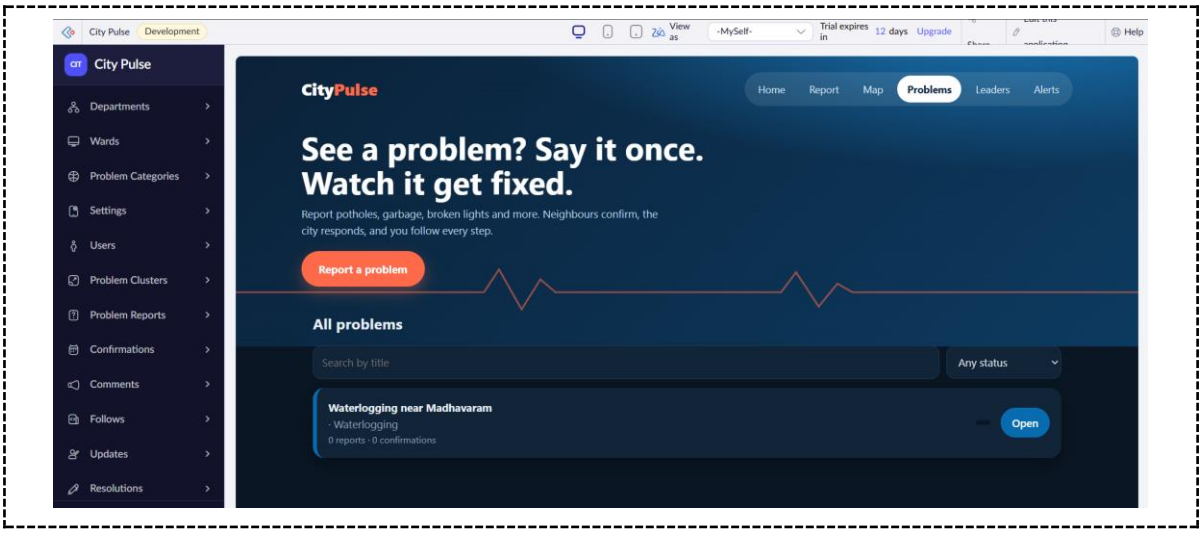


Figure 8.2: *Scenario Step 2 – Automatic Cluster Creation*

8.2 Achievement of Objectives

Objective	Design / deliverable	Assessment
Linked data model	Fifteen forms with lookup fields and a subform.	Covered in the application design.
Automatic clustering	Cluster Report workflow groups nearby reports by category and radius.	Requires execution evidence from the live app.
Assignment and SLA	Ward, department and SLA date assigned on cluster creation; scoring schedule.	Requires test data for wards, departments and categories.
Participation and notifications	Confirmations, comments, follows, points and cp.notifyUser.	Email delivery requires environment testing.
Escalation and closure	SLA Check and Auto Close schedules.	Schedules must be activated and run history attached.
Visibility	Sixteen reports, map view and six pages.	Report and page availability should be verified in Creator.

Table 8.1: Objective Achievement Mapping

8.3 Comparison with the Existing System

Area	Manual process	Proposed City Pulse process
Reporting	Phone calls, letters and social media posts.	Structured report form with category, severity, map location and photos.
Duplicates	Each complaint handled separately.	Nearby same-category reports grouped into one cluster.
Assignment	Manual forwarding to a department.	Ward, department and SLA date assigned automatically.
Prioritisation	By instinct or loudest complaint.	Severity score and priority computed daily.
Escalation	Depends on follow-up by individuals.	Overdue problems flagged and emailed to the department head.
Feedback	Citizen rarely informed.	Notifications on receipt, status change and resolution; citizen verification.
Visibility	Manual consolidation.	Reports, map view and Home page counters.

Table 8.2: Comparison with the Existing System

8.4 Limitations of the Low-Code Approach

The solution depends on platform availability, plan capabilities and the correct configuration of permissions, workflows and schedules. Clustering based on a simple bounding box around coordinates is adequate for a prototype but is not a full geospatial engine. Large datasets can require report and query optimisation. Notification channels such as SMS and push need external services. Low-code does not remove the need for software testing, access-control review, data-quality checks or a deployment process.

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

Visual form, report, map and page builders reduce repetitive coding, while Deluge provides control over clustering, scoring and escalation rules. The project therefore demonstrates a low-code approach rather than a purely no-code solution. A separate pro-code service may be justified in the future for requirements such as image-based classification of problems or advanced geospatial analysis, but should be introduced only when platform-native capabilities are insufficient.

CHAPTER 9

CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

City Pulse presents a structured low-code solution for reporting and managing civic problems. Its forms connect users, wards, departments, categories, reports, clusters, confirmations, comments, follows, updates, resolutions and notifications. The workflow design addresses common operational risks by grouping duplicate reports, assigning responsibility and SLA dates automatically, scoring and escalating problems, informing citizens and closing resolved problems without manual effort.

The report follows the prescribed low-code project structure and documents the requirements, design, implementation modules, testing approach, security considerations and expected outcomes. Before the report is submitted as evidence of a live deployment, the test cases should be executed and the relevant application screenshots, execution logs and deployment details should be added.

9.2 Future Enhancements

- Activate and verify the scheduled workflows and the portal-user workflow in production.
- Enforce the daily report limit and spam-strike limit and automate account suspension.
- Add SMS and push notification delivery through an approved provider.
- Implement merging of duplicate clusters using the Merged Into field.
- Bind the leaderboard to live contribution scores and award badges.
- Add ward-wise dashboards with charts for resolution time, SLA compliance and category trends.
- Provide a Tamil user interface and a mobile app with camera and GPS capture.
- Use image and text analysis to suggest the category and severity of a report.
- Collect measurable results such as duplicate reports merged, average resolution time and SLA breaches escalated.

9.3 Learning Outcomes and Career Relevance

This project provides practical experience in business process analysis, relational data modelling, form design, lookup relationships, customer portals, map reports, event-driven automation, Deluge scripting, scheduled workflows, notifications, report design and test planning. These skills are directly relevant to roles in low-code development, business application development and process automation.

REFERENCES

[1] Zoho Corporation, “Zoho Creator Help Documentation,” Zoho. Available: https://www.zoho.com/creator/help/ (accessed Oct. 31, 2026).

[2] Zoho Corporation, “Deluge Help,” Zoho. Available: https://www.zoho.com/deluge/help/ (accessed Oct. 31, 2026).

[3] Zoho Corporation, “Workflow Rules in Zoho Creator,” Zoho Creator Help. Available: https://www.zoho.com/creator/help/ (accessed Oct. 31, 2026).

[4] Zoho Corporation, “Schedules in Zoho Creator,” Zoho Creator Help. Available: https://www.zoho.com/creator/help/ (accessed Oct. 31, 2026).

[5] Zoho Corporation, “Customer Portal in Zoho Creator,” Zoho Creator Help. Available: https://www.zoho.com/creator/help/ (accessed Oct. 31, 2026).

[6] mySociety, “FixMyStreet,” mySociety. Available: https://www.fixmystreet.com/ (accessed Oct. 31, 2026).

[7] SeeClickFix, “SeeClickFix – Civic Issue Reporting.” Available: https://seeclickfix.com/ (accessed Oct. 31, 2026).

[8] OWASP Foundation, “Application Security Verification Standard (ASVS),” OWASP. Available: https://owasp.org/www-project-application-security-verification-standard/ (accessed Oct. 31, 2026).

APPENDIX A – FEATURE COVERAGE CHECKLIST

Feature area	Specified in project	Evidence to attach / verify
15 forms	Yes	Screenshots of each form in Creator and sample records.
Lookup relationships and subform	Yes	Form builder screenshots and sample linked records.
Deluge validation and clustering scripts	Implemented in form workflows	Script screenshots and test outputs.
Form workflows	15 workflows	Workflow configuration screenshots and execution logs.
Scheduled workflows	3 schedules (inactive in export)	Schedule configuration, activation and run history.
Notification function	cp.notifyUser	Function screenshot and notification records.
Reports	14 list reports and 2 map reports	Report screenshots with representative data.
Pages	6 pages	Page screenshots for Home, Map View, Report a Problem, My Reports and Leaderboard.
Customer portal	citizen, officer, moderator, Customer profiles	Portal profile screenshots and login test.
Deployment	Not evidenced in the export	Published URL, release date and deployment checklist.

Table A.1: Feature Coverage Checklist

APPENDIX B – REPORTS AND PAGES CATALOGUE

B.1 Reports

- All Departments
- All Wards
- All Problem Categories
- All Settings
- All Users
- All Problem Clusters
- All Problem Reports
- All Confirmations
- All Comments
- Follows Report
- All Updates
- All Resolutions
- All Notifications
- All Announcements
- Flags Report
- Problem Clusters Map

B.2 Pages

- Home
- Map View
- Report a Problem
- My Reports
- Leaderboard
- City Pulse (widget page)

B.3 Home Page Indicators

- Open Civic Issues – count of problem clusters whose status is not Resolved or Closed.

- Issues Resolved – count of problem clusters whose status is Resolved or Closed.

B.4 Scheduled Monitoring

Frequency	Monitoring activities
Daily	Compute persistence days, severity score and priority; promote Emerging problems at the threshold; flag overdue problems and email the department head; close problems resolved for longer than the configured days.
Weekly	Review overdue and high-priority problems; review ward-wise open problems; review flagged items.
Monthly	Review resolution times and SLA compliance by department; review categories and settings; review top contributors.

Table B.1: Scheduled Monitoring

APPENDIX C – DELUGE LOGIC SUMMARY

The following pseudocode summaries describe the behaviour of the implemented scripts. They are not substitutes for the exact field link names and deployable Deluge source code in the live Creator application.

C.1 Cluster Report (Problem Reports – on success)

Read the cluster radius and points per report from Settings. Read the category name, department, SLA days and any category radius. Find the nearest active ward using squared distance between the report and each ward centre. Look for an open cluster of the same category whose latitude and longitude lie within the radius box; if found, add one to its report count and update the last-reported time. Otherwise insert a new cluster titled “category near ward” with status Emerging and SLA due date equal to today plus SLA days. Link the report to the cluster and ward, add points and a report to the reporter, and call the notification function.

C.2 Coordinates and duplicate checks

On validate, read latitude and longitude from the Address field; if either is empty, alert the user and cancel submission. For a confirmation, count confirmations by the same user on the same cluster today and cancel if one exists. For a follow, count existing follows by the same user and cluster and cancel if one exists.

C.3 Counters and points

After a confirmation, add one to the cluster’s confirmation count, update last-reported time, reopen the cluster when appropriate and add the configured points to the user. After a comment, add one to the comment count and add points. After a follow, add one to the follower count.

C.4 Update and resolution

On validate of an update, set the updated-by user, store the current status as the previous status and reject any new status outside the seven allowed values. On success, set the cluster status, store verified details when it becomes Active, store the resolved time when it becomes Resolved, and notify every reporter and every active follower. On a resolution, set the cluster to Resolved, store the resolved time, insert a public update and notify reporters.

C.5 Scheduled workflows

Update Scores: for each open cluster compute days open and severity score as weight × reports × (confirmations + 1) × (days + 1), set priority from the score and promote Emerging clusters at the threshold. SLA Check: for each open cluster whose SLA due date has passed and which is not yet overdue, set overdue, raise the escalation level and email the department head, copying the administrator. Auto Close: close clusters that have been Resolved for longer than the configured number of days.

C.6 Notification function

The function cp.notifyUser(userId, clusterId, type, message) inserts a Notification record for the user on the In-App channel and, when the user's email notification flag is not false and an email address exists, sends an email with the subject "CityPulse:" followed by the notification type.

APPENDIX D – USER MANUAL

D.1 General Access

1. Open the authorised City Pulse portal URL using a supported browser.
2. Sign in with the account provided or register as a citizen and accept the terms.
3. Use the navigation menu to open the page, form or report appropriate to your role.
4. Enter mandatory fields, verify selections and save the record.
5. Use the status shown on reports to monitor progress; do not create duplicate records to retry a failed action.

D.2 Citizen

6. Open Report a Problem, choose a category and describe the problem.
7. Pick the location on the map, set the severity and attach photos.
8. Submit the report and note the notification you receive.
9. Open My Reports to follow the status of your reports.
10. Open View Problems, then confirm, comment on or follow problems of interest.

D.3 Moderator

11. Review new reports and set the status Under Review, Verified or Rejected, with a rejection reason where needed.
12. Review flagged reports, comments and users and update the flag status.

D.4 Officer

13. Open the Problem Clusters report and select a problem assigned to your department.
14. Post an update with the new status and a short note.
15. When work is complete, submit a resolution with a description and before and after photographs.

D.5 Administrator

16. Maintain wards (with centre coordinates), departments (with head email), categories (with SLA days and weights) and Settings.

17. Review reports, the map and the Home page counters; review portal profile permissions periodically.
18. Test workflow changes in a safe environment before publishing.

D.6 Troubleshooting

Issue	Recommended check
Report not submitted	Confirm that a map location was chosen and that mandatory fields are filled.
Report did not join an existing cluster	Check the category, the distance between the reports, the cluster radius and the existing cluster status.
No notification received	Check the user's notification preferences, email address, workflow status and execution history.
Overdue problem not escalated	Check that the SLA Check schedule is active and the cluster has an SLA due date and department.
Status not changing	Check that the new status is one of the allowed values and the update workflow is active.
Home counters look wrong	Check the status values of the problem clusters and the counter criteria.

Table D.1: Troubleshooting

APPENDIX E – EVALUATION RUBRIC

Student Name	S BALAJI	**Register No.**	2116240701069
Project Title	City Pulse – Civic Issue Reporting and Tracking System	**Date**	

S.No.	**Criterion**	**Indicators**	**Max. Marks**	**Marks Awarded**
1	Problem Definition & Requirement Analysis	Clarity of problem, measurable objectives, complete FR/NFR, use cases	10	
2	System Design	Architecture, ER/data model, form specifications, workflow design, permission matrix	10	
3	Forms, Reports & Pages	Variety of field types, lookups/subforms, report types, UI Builder dashboards	15	
4	Deluge Scripting & Workflows	Correctness and depth of scripts; form, schedule, approval, Blueprint, batch workflows	15	
5	Integration, RBAC & AI	Zoho/external integrations, connections, roles & permissions, Zia/analytics	10	
6	Testing, Deployment, Security & Governance	Test coverage, environments, optimisation, security controls, audit	10	
7	Live Demonstration	End-to-end working of the application across roles; feature checklist verified	15	
8	Project Report	Adherence to template and format, clarity, figures, references	5	
9	Viva-Voce	Understanding of the platform, design decisions and low-code concepts	10	
	TOTAL		100	

Table E.1: Evaluation Rubric for the Individual Project

Level	**% of Max. Marks**	**Descriptor**
Excellent	90–100	Feature implemented completely, correctly and beyond the basic requirement; student explains design choices confidently.
Good	70–89	Feature implemented correctly with minor gaps; explanation adequate.

Level	% of Max. Marks	Descriptor
Satisfactory	50–69	Feature partially implemented or works only for the main path; limited explanation.
Needs Improvement	< 50	Feature missing, non-functional or not understood by the student.

Table E.2: Performance Level Descriptors

Signature of Internal Examiner
Name: _____

Signature of External Examiner
Name: _____

APPENDIX F – VIVA-VOCE RECORD SHEET

Student Name	S BALAJI	**Register No.**	2116240701069
Department / Section	Computer Science And Engineering	**Date of Viva**	
Application URL Verified	Yes / No	**Demo Duration**	_____ min

Q.No.	**Question / Topic Asked**	**Unit**	**Response (E / G / S / NI)**	**Remarks**
1				
2				
3				
4				
5				
6				
7				
8				

Table F.1: Viva-Voce Questions and Responses

E – Excellent G – Good S – Satisfactory NI – Needs Improvement

Component	**Max. Marks**	**Marks Awarded**
Criteria 1–8 (Appendix E)	90	
Viva-Voce	10	
TOTAL	100	

Table F.2: Marks Summary

Examiners' Remarks:

INTERNAL EXAMINER

Name: _____

EXTERNAL EXAMINER

Name: _____